

API

FORMULA 1



ALEJANDRA O'SHEA, MARTA SANCHEZ, PILAR BOURG
IGNACIO LORÉN, GABRIEL LAZOVSKY, JAVIER ROZALÉN, JAVIER DE QUADROS



“The dataset consists of all information on the Formula 1 races, drivers, constructors, qualifying, circuits, lap times, pit stops, championships from 1950 till the latest 2024 season.”

Formula 1 World Championship (1950 - 2024)

▲ 2276

<> Code

Download

Data Card Code (158) Discussion (48) Suggestions (0)

driver_standings.csv (883.77 kB)

Detail Compact Column

7 of 7 columns

About this file

Final standings of the driver's championship

driverStandingsId Id	raceId Id of race	driverId Id of driver	# points Points	# position Final position	# position Final position
1	1	1	0	1	1
2	18	2	10	1	1
3	18	3	8	2	2
4	18	4	6	3	3
5	18	5	5	4	4
6	18	6	4	5	5
7	18	7	3	6	6
8	18	8	2	7	7
9	18	9	1	8	8
10	19	1	14	1	1
11	19	2	11	3	3

Data Explorer

Version 24 (21.99 MB)

circuits.csv

constructor_results.csv

constructor_standings.csv

constructors.csv

driver_standings.csv

drivers.csv

lap_times.csv

pit_stops.csv

qualifying.csv

races.csv

results.csv

seasons.csv

sprint_results.csv

status.csv

Summary

14 files

120 columns



```
import { MongoClient } from "mongodb";
import fs from "fs";
import csv from "csv-parser";

const uri = "mongodb://127.0.0.1:27017";
const client = new MongoClient(uri);

const DATABASE_NAME = "f1_api";

const importCSV = async (filePath, collectionName) => {
  ...
};

const seedDatabase = async () => {
  try {
    await client.connect();

    console.log("MongoDB connected");

    await importCSV("./data/drivers.csv", "drivers");
    await importCSV("./data/races.csv", "races");
    await importCSV("./data/circuits.csv", "circuits");
    await importCSV("./data/results.csv", "results");

    console.log("Database seeded successfully");

    process.exit();
  } catch (error) {
    console.error(error);

    process.exit(1);
  }
};

seedDatabase();
```

Collection name ⚙	Properties ⚙	Storage size ⚙	Data size ⚙	Documents ⚙	Avg. document size ⚙	Indexes ⚙	Total index size
Circuits	-	45.06 kB	17.85 kB	77	231.00 B	1	36.86 kB
ConstructorResults	-	585.73 kB	1.34 MB	13K	106.00 B	1	548.86 kB
ConstructorStandings	-	880.64 kB	1.81 MB	13K	135.00 B	1	581.63 kB
Constructors	-	53.25 kB	35.55 kB	212	167.00 B	1	40.96 kB
DriverStandings	-	2.20 MB	4.36 MB	35K	125.00 B	1	1.47 MB
Drivers	-	122.88 kB	190.48 kB	861	221.00 B	1	49.15 kB
LapTimes	-	21.06 MB	63.62 MB	589K	108.00 B	1	17.54 MB
PitStops	-	929.79 kB	1.39 MB	11K	122.00 B	1	454.66 kB
Qualifying	-	897.02 kB	1.60 MB	10K	152.00 B	1	425.98 kB
Races	-	180.22 kB	420.44 kB	1.1K	373.00 B	1	69.63 kB
Results	-	3.68 MB	8.19 MB	27K	306.00 B	1	1.05 MB
Seasons	-	36.86 kB	7.09 kB	75	94.00 B	1	36.86 kB

```
driverId,driverRef,number,code,forename,surname,dob,nationality,url
1,"hamilton",44,"HAM","Lewis","Hamilton","1985-01-07","British","http://en.wikipedia.org/wiki/Lewis_Hamilton"
2,"heidfeld","\N","HEI","Nick","Heidfeld","1977-05-10","German","http://en.wikipedia.org/wiki/Nick_Heidfeld"
3,"rosberg",6,"ROS","Nico","Rosberg","1985-06-27","German","http://en.wikipedia.org/wiki/Nico_Rosberg"
4,"alonso",14,"ALO","Fernando","Alonso","1981-07-29","Spanish","http://en.wikipedia.org/wiki/Fernando_Alonso"
5,"kovalainen","\N","KOV","Heikki","Kovalainen","1981-10-19","Finnish","http://en.wikipedia.org/wiki/Heikki_Kovalainen"
```



Formula 1 API

openapi: 3.0.3

info: {3 keys}

tags: [10 items]

paths: {31 keys}

components: {3 keys}

tags[0]

name: Drivers

tags[1]

name: Constructors

tags[2]

name: Circuits

tags[3]

name: Races

tags[4]

name: Seasons

tags[5]

name: Results

tags[6]

name: Standings

tags[7]

name: Qualifying

tags[8]

name: Lap Times

tags[9]

name: Pit Stops

paths

/drivers: {2 keys}

/drivers/{driverId}: {4 keys}

/drivers/{driverId}/results: {2 keys}

/drivers/{driverId}/standings: {2 keys}

/drivers/{driverId}/qualifying: {2 keys}

/drivers/{driverId}/lap-times: {2 keys}

/drivers/{driverId}/pit-stops: {2 keys}

/constructors: {2 keys}

/constructors/{constructorId}: {4 keys}

/constructors/{constructorId}/results: {2 keys}

/constructors/{constructorId}/standings: {2 keys}

/circuits: {2 keys}

/circuits/{circuitId}: {4 keys}

/circuits/{circuitId}/races: {2 keys}

/races: {2 keys}

/races/{raceId}: {4 keys}

/races/{raceId}/results: {2 keys}

/races/{raceId}/qualifying: {2 keys}

/seasons: {1 keys}

/seasons/{year}: {2 keys}

/seasons/{year}/races: {2 keys}

/results: {1 keys}

/results/{resultId}: {2 keys}

/standings/drivers: {1 keys}

/standings/constructors: {1 keys}

/qualifying: {1 keys}

/qualifying/{qualifyId}: {2 keys}

/lap-times: {1 keys}

/pit-stops: {1 keys}

/external/current-season: {1 keys}

/external/driver/{driverRef}: {1 keys}

/drivers

get: {5 keys}

post: {4 keys}

get

tags: [1 items]

summary: Get all drivers

description: Returns a list of drivers with optional filters.

parameters: [9 items]

responses: {3 keys}

parameters[3]

name: forename

in: query

schema: {1 keys}

description: Filter by forename (partial match)

parameters[4]

name: surname

in: query

schema: {1 keys}

description: Filter by surname (partial match)

parameters[5]

name: dob

in: query

schema: {2 keys}

description: Filter by date of birth



```
/drivers:
  get:
    tags:
      - Drivers
    summary: Get all drivers
    description: Returns a list of drivers with optional filters.
    parameters:
      - name: driverRef
        in: query
        schema:
          type: string
          description: Filter by driver reference
        ...
    responses:
      '200':
        description: List of drivers
        content:
          application/json:
            schema:
              type: array
              items:
                $ref: '#/components/schemas/Driver'
      '400':
        $ref: '#/components/responses/BadRequest'
      '500':
        $ref: '#/components/responses/InternalServerError'

  post:
    tags:
      - Drivers
    summary: Create a new driver
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/DriverInput'
          example:
            driverRef: hamilton
            number: 44
            code: HAM
            forename: Lewis
            surname: Hamilton
            dob: "1985-01-07"
            nationality: British
            url: "https://en.wikipedia.org/wiki/Lewis_Hamilton"
    responses:
      '201':
        description: Driver created successfully
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Driver'
      '400':
        $ref: '#/components/responses/BadRequest'
      '500':
        $ref: '#/components/responses/InternalServerError'
```

Drivers

GET	/drivers	Get all drivers	⌵
POST	/drivers	Create a new driver	⌵
GET	/drivers/{driverId}	Get a specific driver by ID	⌵
PUT	/drivers/{driverId}	Update an existing driver	⌵
DELETE	/drivers/{driverId}	Delete a driver	⌵
GET	/drivers/{driverId}/results	Get race results for a driver	⌵
GET	/drivers/{driverId}/standings	Get championship standings for a driver	⌵
GET	/drivers/{driverId}/qualifying	Get qualifying results for a driver	⌵
GET	/drivers/{driverId}/lap-times	Get lap times for a driver	⌵



driverRoutes.mjs

```
import express from "express";
import {
  getAllDrivers,
  createDriver,
  getDriverById,
  updateDriver,
  deleteDriver,
  getDriverResults,
  getDriverStandings,
  getDriverQualifying,
  getDriverLapTimes,
  getDriverPitStops,
} from "../controllers/driverController.mjs";

const router = express.Router();

router.get("/", getAllDrivers);
router.post("/", createDriver);
router.get("/:driverId", getDriverById);
router.put("/:driverId", updateDriver);
router.delete("/:driverId", deleteDriver);
router.get("/:driverId/results", getDriverResults);
router.get("/:driverId/standings", getDriverStandings);
router.get("/:driverId/qualifying", getDriverQualifying);
router.get("/:driverId/lap-times", getDriverLapTimes);
router.get("/:driverId/pit-stops", getDriverPitStops);

export default router;
```

driverController.mjs

```
import db from "../db/conn.mjs";
import { sendResponse } from "../utils/sendResponse.mjs";
import Ajv from "ajv";
import addFormats from "ajv-formats";

const ajv = new Ajv();
addFormats(ajv);

const driverSchema = {
  type: "object",
  properties: {
    driverId: { type: "integer" },
    driverRef: { type: "string" },
    number: { type: ["integer", "string", "null"] },
    code: { type: ["string", "null"] },
    forename: { type: "string" },
    surname: { type: "string" },
    dob: { type: "string", format: "date" },
    nationality: { type: "string" },
    url: { type: "string", format: "uri" },
  },
  required: ["driverRef", "forename", "surname", "nationality"],
  additionalProperties: false,
};

const driverUpdateSchema = {
  ...driverSchema,
  required: [],
};
```

```
export const getAllDrivers = async (req, res) => {
  try {
    const query = {};
    if (req.query.nationality) {
      query.nationality = req.query.nationality;
    }

    const drivers = await
    db.collection("Drivers").find(query).toArray();

    if (drivers.length === 0) {
      return sendResponse(req, res, 404, {
        status: 404, message: "No drivers found" },
        "error");
    }

    sendResponse(req, res, 200, drivers,
    "drivers", "driver");
  } catch (error) {
    res
      .status(500)
      .json({ status: 500, message: "Server
error", error: error.message });
  }
};
```



DEMO

DEMOSTRACIÓN

